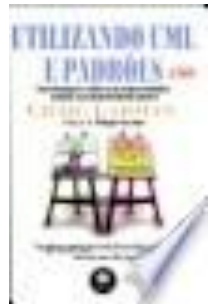


Utilizando UML e Padrões

Capítulo 2 – Iterativo, Evolutivo e Ágil



Objetivos

- Fornecer motivação para o conteúdo e ordem das próximas aulas
- Definir um **processo iterativo e ágil**
- Definir conceitos fundamentais do **Processo Unificado**

O que vem a seguir?



Introdução

- **Desenvolvimento iterativo e evolutivo**
 - Envolve a **imediata programação** e **teste** de um sistema parcial em ciclos repetidos
 - Desenvolvimento começa **antes** que os requisitos tenham definidos em detalhe.
 - A **realimentação** é usada para esclarecer e aperfeiçoar as especificações em evolução.

Introdução

- **Modelo de ciclo de vida em cascata**
 - Promove **passos grandes de requisitos e projeto especulativos** logo no início, antes da programação.
 - Diversos estudos demonstram que o **ciclo em cascata** é fortemente associado às maiores taxas de **falhas em projetos de software**.

O que é o PU? os outros métodos são complementares?

- **Processo de desenvolvimento de software**
 - Descreve uma abordagem para a **construção, implantação e manutenção** do software.
- O **PU** é um exemplo de um **processo iterativo** para o desenvolvimento de software orientados a objetos.
 - O **RUP**, um refinamento detalhado do **PU** é muito adotado na prática.

O que é o PU? Os outros métodos são complementares?

- Este **curso** terá como base o **PU**
- **Razões:**
 - **Flexível e aberto**
 - Incentiva a inclusão de práticas de outros métodos iterativos
 - **XP, Scrum, etc.**
 - **Exemplo:** práticas do **XP** de **desenvolvimento guiado por testes, refatoração e integração contínua** podem ser usadas em um **projeto PU**.

E se o Processo Unificado não me interessar?

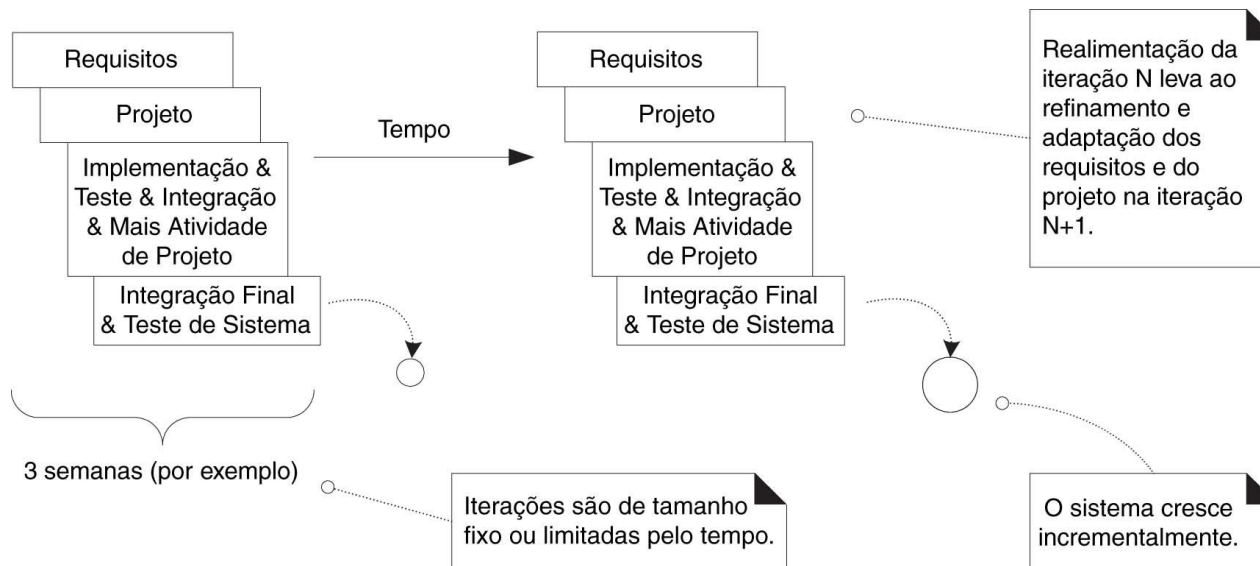
- O **PU** é usado neste curso como um processo exemplo
 - **A/POO** iterativos são ensinados no contexto de um **processo**.
- As idéias centrais deste curso são **independentes de processo** em particular:
 - Como **pensar** e **projetar** em termos de **objetos**
 - Aplicar **UML**
 - Usar **padrões de projeto**
 - **Modelagem ágil**
 - **Análise de requisitos evolutiva**
 - Escrever **casos de uso**
 - **Projeto de software**

E se o Processo Unificado não me interessar?

- Muitas destas técnicas são usadas em métodos modernos **iterativos, evolutivos e ágeis**
 - **Scrum, Lean, DSDM, Feature Driven Development, Adaptative Software Development** e outros.

O que é desenvolvimento iterativo e evolutivo?

- Também conhecida como **desenvolvimento iterativo e incremental** ou **desenvolvimento evolutivo e em espiral**



O que é desenvolvimento iterativo e evolutivo?

- O resultado de cada **iteração** é um **sistema executável**
 - Porém, incompleto
 - Talvez sejam necessárias várias **iterações** para colocarmos o sistema em **ambiente de produção**.

O que é desenvolvimento iterativo e evolutivo?

- A saída de uma **iteração** não é um **protótipo descartável**
 - **Desenvolvimento iterativo** não é **prototipação**
 - A saída é um subconjunto do sistema com **qualidade final de produção**.

O que é desenvolvimento iterativo e evolutivo?

- Como acolher **mudanças** em um projeto **iterativo**?
 - *“Embrace the Change”*, subtítulo do famoso livro sobre **XP** do **Kent Beck**
 - Enfatiza a atitude-chave no desenvolvimento iterativo:
 - A atitude de **aceitar mudanças e adaptação** como fatores inevitáveis e essenciais.

O que é desenvolvimento iterativo e evolutivo?

- Como acolher **mudanças** em um projeto **iterativo**? (cont.)
 - O **PU** equilibra as necessidades de estabilização e flexibilização de requisitos.
 - Cada **iteração** foca em um subconjunto dos **requisitos** que são entregues o quanto antes para o cliente
 - Essa **realimentação** rápida fornece a percepção crucial para **modificar** ou **adaptar** a compreensão dos **requisitos** ou do projeto.

O que é desenvolvimento iterativo e evolutivo?

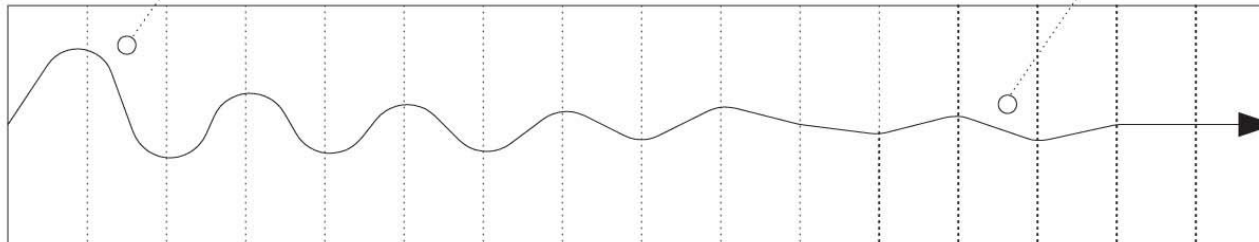
- Como acolher **mudanças** em um **projeto iterativo**?
(cont.)
 - Além de esclarecer requisitos, atividades como **testes de carga** provarão se o projeto e implementação parciais estão no caminho certo
 - Senão, deveremos mudar a **arquitetura central** do sistema nas próximas iterações
 - Conseqüentemente, o trabalho progride por meio de uma série de **ciclos estruturados** em **construção-Realimentação-Adaptação**

O que é desenvolvimento iterativo e evolutivo?

- A **instabilidade dos requisitos** e do **projeto** diminui com o tempo

Iterações iniciais estão longe do "verdadeiro caminho" do sistema. Por meio de realimentação e adaptação, o sistema converge para os requisitos e o projeto mais adequados.

Nas últimas iterações, é rara uma mudança significativa de requisitos, mas ela pode ocorrer. Essas mudanças tardias podem dar à organização uma vantagem competitiva nos negócios.



uma iteração de projeto,
implementação, integração e teste

Existem benefícios no desenvolvimento iterativo?

- **Menos erros de projeto, maior produtividade e menor taxa de defeitos**
 - Comprovados por pesquisas sobre **métodos iterativos e evolutivos**
- Minimização precoce de altos **riscos**
- Visibilidade para o cliente do **status** do projeto desde o início

Existem benefícios no desenvolvimento iterativo?

- **Realimentação, envolvimento do usuário e adaptação** imediatos
- A equipe não é contagiada pela **paralisa da análise**
- **Lições aprendidas** aplicadas em iterações subsequentes.

Quão longa deve ser uma iteração? O que são limites temporais?

- A maioria dos **métodos iterativos** recomenda que a duração de uma **iteração** seja entre **duas e seis semanas**.
- **Iterações** tem limites temporais
 - Duração fixa
 - No final da iteração o sistema parcial deve estar **testado e integrado**.
 - Perca funcionalidades, mas não perca datas !

O que dizer do ciclo de vida em cascata?

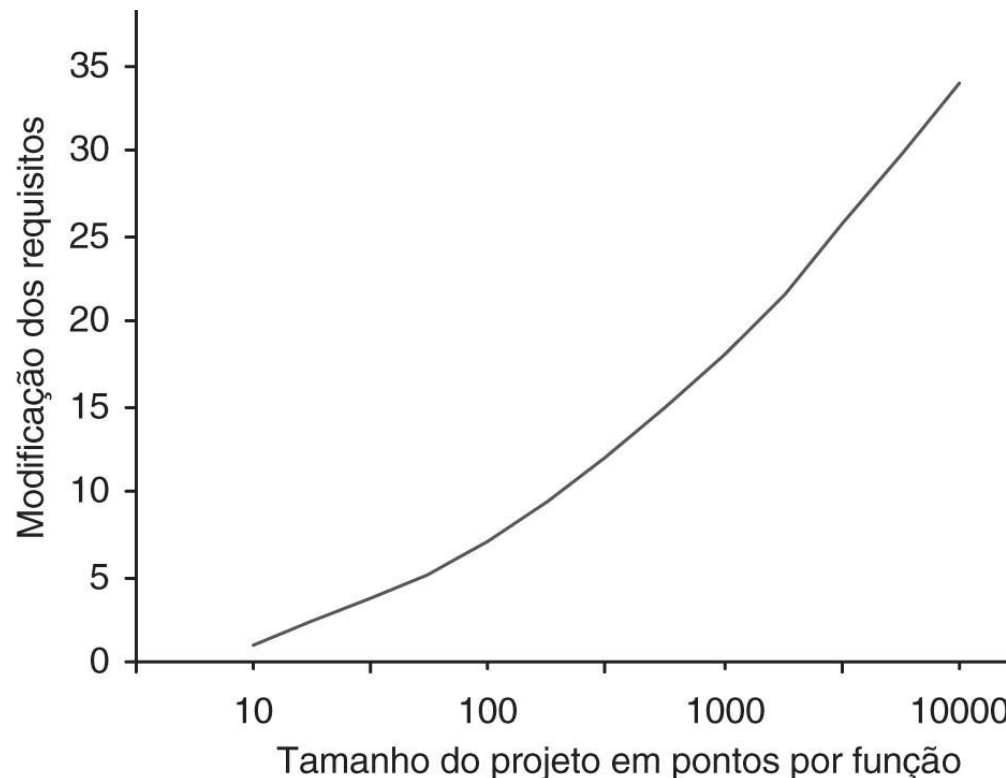
- Tentativa em **detalhar** a maioria dos **requisitos** antes da programação
 - Da mesma forma, criar um conjunto de modelos de **projeto** antes da **programação**
 - Idem, criar um **cronograma confiável** logo no **começo do projeto**.

O que dizer do ciclo de vida em cascata?

- **Diretriz:** não deixe o **raciocínio em cascata** invadir um **projeto iterativo ou PU**
 - Escrever **todos os casos de uso em detalhes** antes da programação
 - Fazer **vários modelos OO detalhados** antes de começar a programar.

Por que o ciclo em cascata é tão propenso à falha?

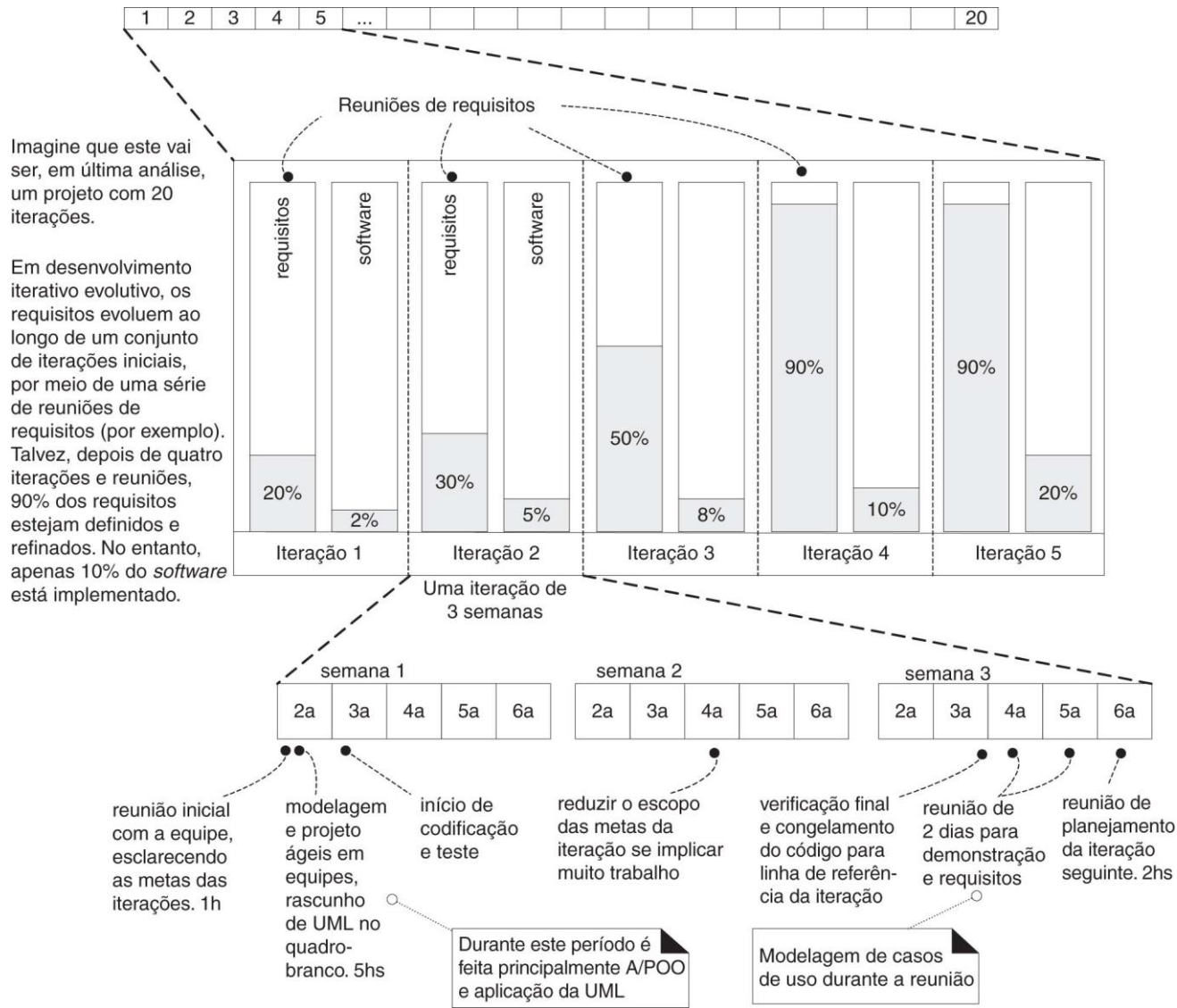
Modificação é uma constante em projetos de software. **Métodos iterativos e evolutivos** assumem e **acolhem modificação e adaptação** baseados em **realimentação**



Como fazer análise e projeto iterativos e evolutivos?

- Cuidado com o mal entendido de que nada vale a **análise e projeto antes da programação**
 - Mal entendido tão extremo quanto achar que uma **análise antecipada completa** é racional.
 - Existe um **meio termo**.

Como fazer análise e projeto iterativos e evolutivos?



O que é planejamento iterativo guiado por risco e guiado pelo cliente?

- O **PU** se baseia em **planejamento iterativo** guiado por **risco** e pelo **cliente**.
- **Guiado pelo cliente:**
 - Construir características que agreguem valor ao cliente
- **Guiado por risco**
 - **Desenvolvimento iterativo** baseado na **arquitetura**.
 - Não dispor de uma **arquitetura central** e sólida é um **risco alto** e comum

Iterações do curso vs. Iterações de um projeto real

- A **iteração 1** dos **estudos de caso** deste curso é guiada por metas de aprendizado
 - Não é **centrada na arquitetura**.
 - Em um projeto real, trabalharíamos nas coisas difíceis e arriscadas primeiro.

O que são métodos e atitudes ágeis?

- Não há definição precisa, pois as práticas específicas variam muito
- Porém, existem algumas características comuns aos **métodos ágeis**
 - Iterações curtas
 - Refinamento evolutivo de planos, requisitos e projeto.

O que são métodos e atitudes ágeis?

- **Práticas do método ágil Scrum:**
 - Sala de trabalho comum
 - Equipes auto-gerenciáveis que se coordenam por meio de reunião diária.
- **Práticas do método Extreme Programming (XP):**
 - Programação em pares
 - Desenvolvimento guiado por testes.
- O **PU** é flexível pode incluir práticas do **Scrum**, **XP** e outros **métodos**.

O manifesto ágil

O Manifesto Ágil	
Indivíduos e interação	Mais que processos e ferramentas
Software funcionando	Mais que documentação abrangente
Colaboração com o cliente	Mais que negociação contratual
Responder à mudanças	Mais que seguir um plano

O manifesto ágil

Kent Beck	Mike Beedle	Arie van Bennekum
Alistair Cockburn	Martin Fowler	Ken Schwaber
James Grenning	Jim Highsmith	Andrew Hunt
Ron Jeffries	Jon Kern	Brian Marick
Robert C. Martin	Steve Mellor	Ward Cunningham
Jeff Sutherland	Dave Thomas	

Os princípios ágeis

*1 - Nossa maior prioridade é satisfazer o cliente, através da **entrega adiantada e contínua de software de valor.***

Os princípios ágeis

*2 - Aceitar **mudanças de requisitos**, mesmo no fim do desenvolvimento. Processos ágeis se adequam às mudanças, para que o cliente possa tirar vantagens competitivas.*

Os princípios ágeis

*3 - Entregar **software funcionando com frequência**, na escala de poucas semanas ou meses, com preferência aos períodos mais curtos.*

Os princípios ágeis

*4 - Pessoas relacionadas à **negócios e desenvolvedores** devem **trabalhar em conjunto e diariamente**, durante todo o curso do projeto.*

Os princípios ágeis

*5 - Construir projetos ao redor de **indivíduos motivados**. Dando a eles o ambiente e suporte necessários e confiar que farão seu trabalho.*

Os princípios ágeis

*6 - O Método mais eficiente e eficaz de transmitir informações de e para um time de desenvolvimento, é através de **conversa frente a frente**.*

Os princípios ágeis

*7 - Software funcionando é a principal **medida de progresso.***

Os princípios ágeis

*8 - **Processos ágeis** promovem um ambiente sustentável. Os patrocinadores, desenvolvedores e usuários, devem ser capazes de manter um **ritmo constante** indefinidamente.*

Os princípios ágeis

*9 - Contínua atenção à **excelência técnica e bom design**, aumenta a agilidade.*

Os princípios ágeis

*10 - **Simplicidade**: a arte de maximizar a quantidade de trabalho que não precisa ser feito.*

Os princípios ágeis

*11 - As melhores **arquiteturas, requisitos e designs** emergem de **times auto-organizáveis**.*

Os princípios ágeis

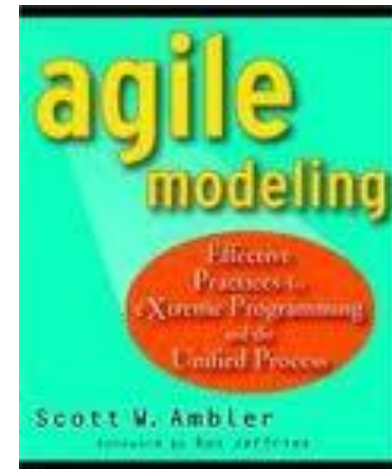
*12 - Em intervalos regulares, o time **reflete** em como ficar mais efetivo, então, ajusta e otimiza seu comportamento de acordo.*

O que é modelagem ágil?

“A finalidade da modelagem é principalmente **entender** e não **documentar**”.

O que é modelagem ágil?

- Esta visão da finalidade da modelagem é conhecida como **modelagem ágil**.



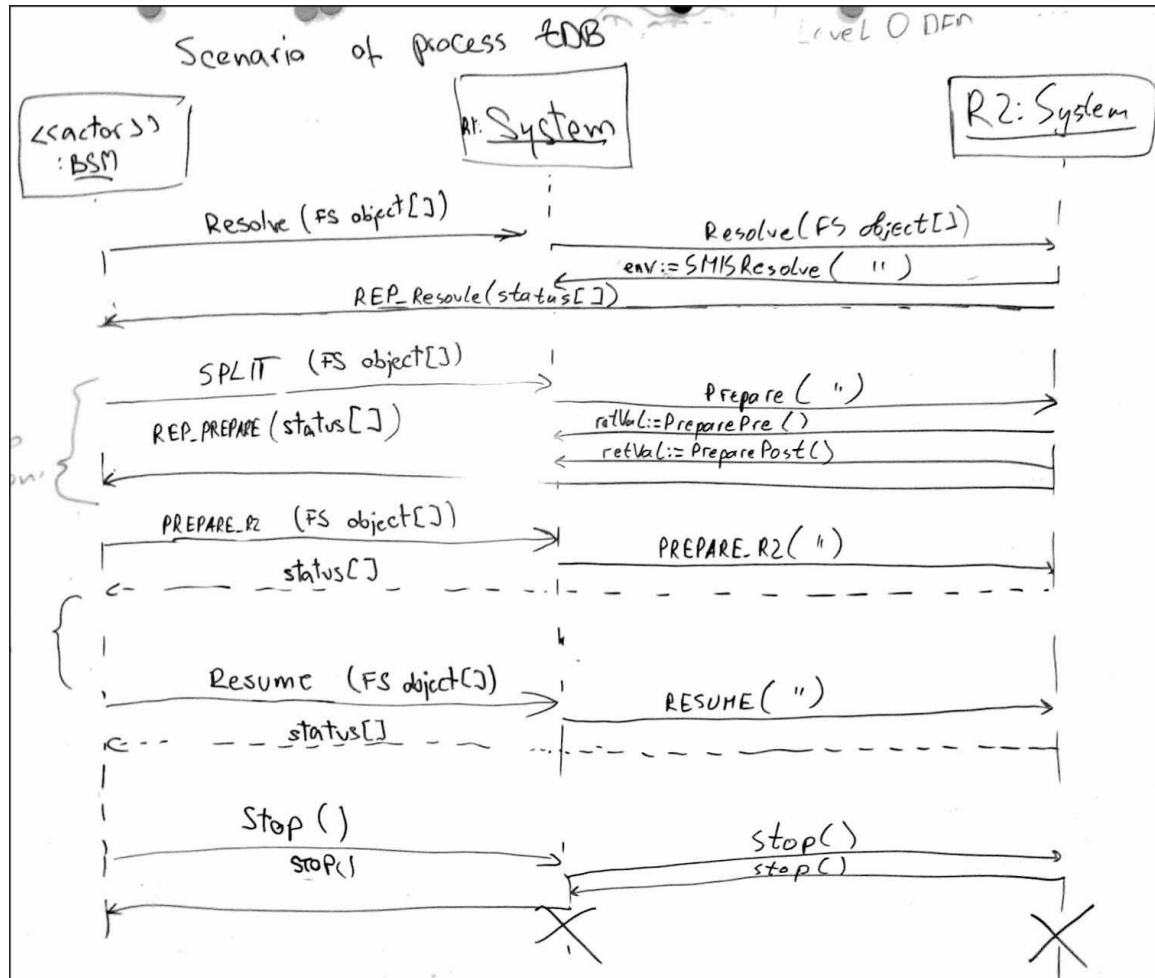
O que é modelagem ágil?

- **Práticas e valores na modelagem ágil:**
 - A adoção de um **método ágil** não significa evitar a modelagem.
 - Modelos para **entendimento e comunicação** e não para documentação
 - Não modele ou aplique **UML** à todo ou à maioria do projeto de software.
 - Use a **ferramenta** mais simples possível

O que é modelagem ágil?

- **Práticas e valores na modelagem ágil (cont):**
 - Não modele sozinho
 - Crie **modelos em paralelos**
 - Use **notação UML simples**
 - Os **modelos** vão ficar imprecisos e o **código** ficará diferente.
 - Os diagramas devem ser criados pelos desenvolvedores para si próprios

Fotografar os rascunhos UML?



O que é um PU ágil?

- O grande conjunto de **artefatos opcionais** previstos no **PU** fez com que muitos pensassem erroneamente que o PU é um **processo “pesado”**.
- Na prática, customizamos o PU, podendo inclusive gerar um **“PU Ágil”**
 - **Exemplo: OpenUP/Basic**

O que é um PU ágil?

- **Uma abordagem ágil para o PU:**
 - Prefira um conjunto pequeno de artefatos
 - **Requisitos** e **projeto** não são completados antes da programação
 - UML com **modelagem ágil**
 - Não há um plano detalhado para todo o projeto.
 - **Plano de fase**
 - **Plano de iteração**

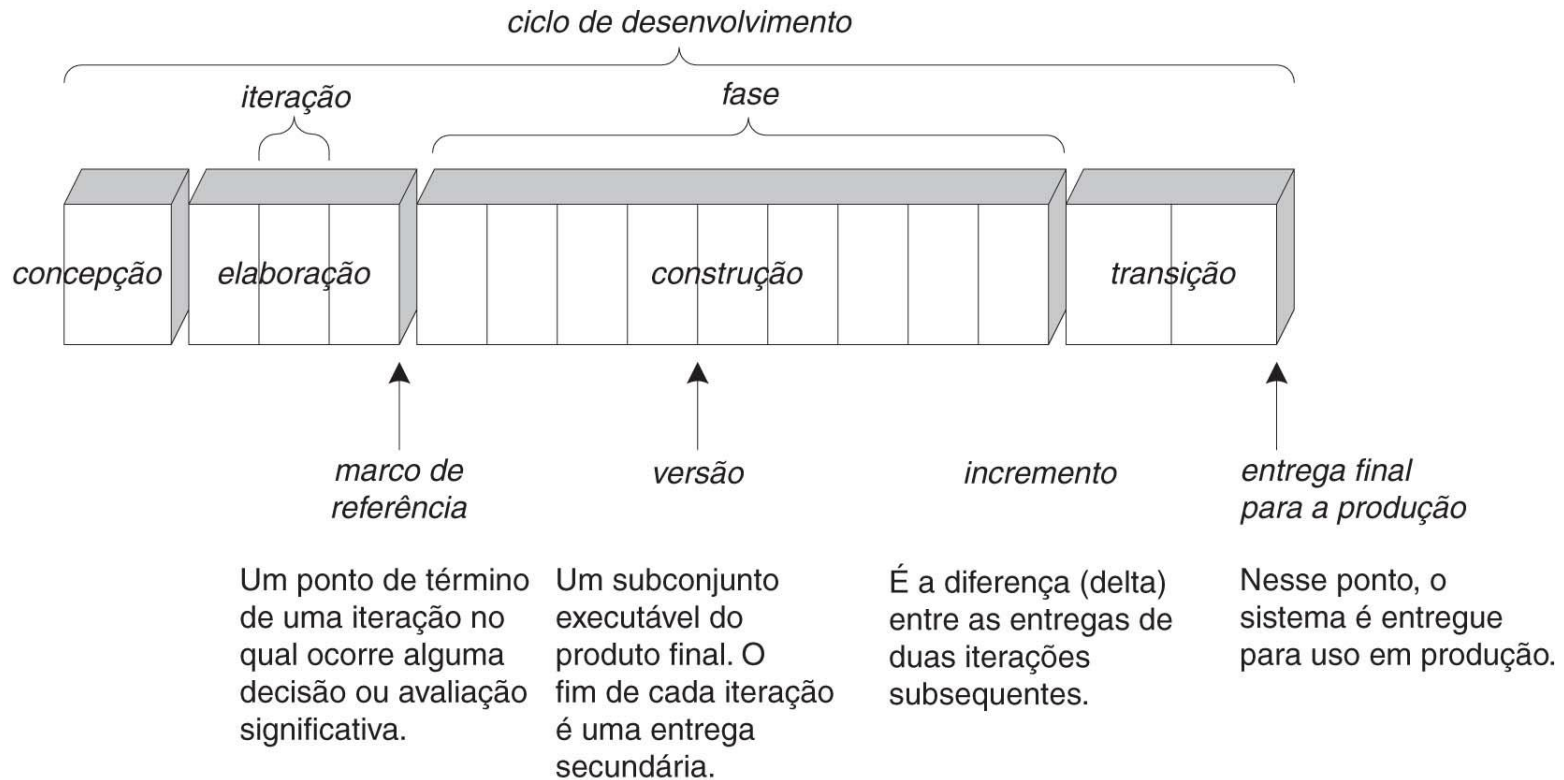
Há outras práticas importantes do Processo Unificado?

- **Idéia central**
 - Iterações curtas, com duração fixa, em um processo de desenvolvimento iterativo, evolutivo e adaptativo

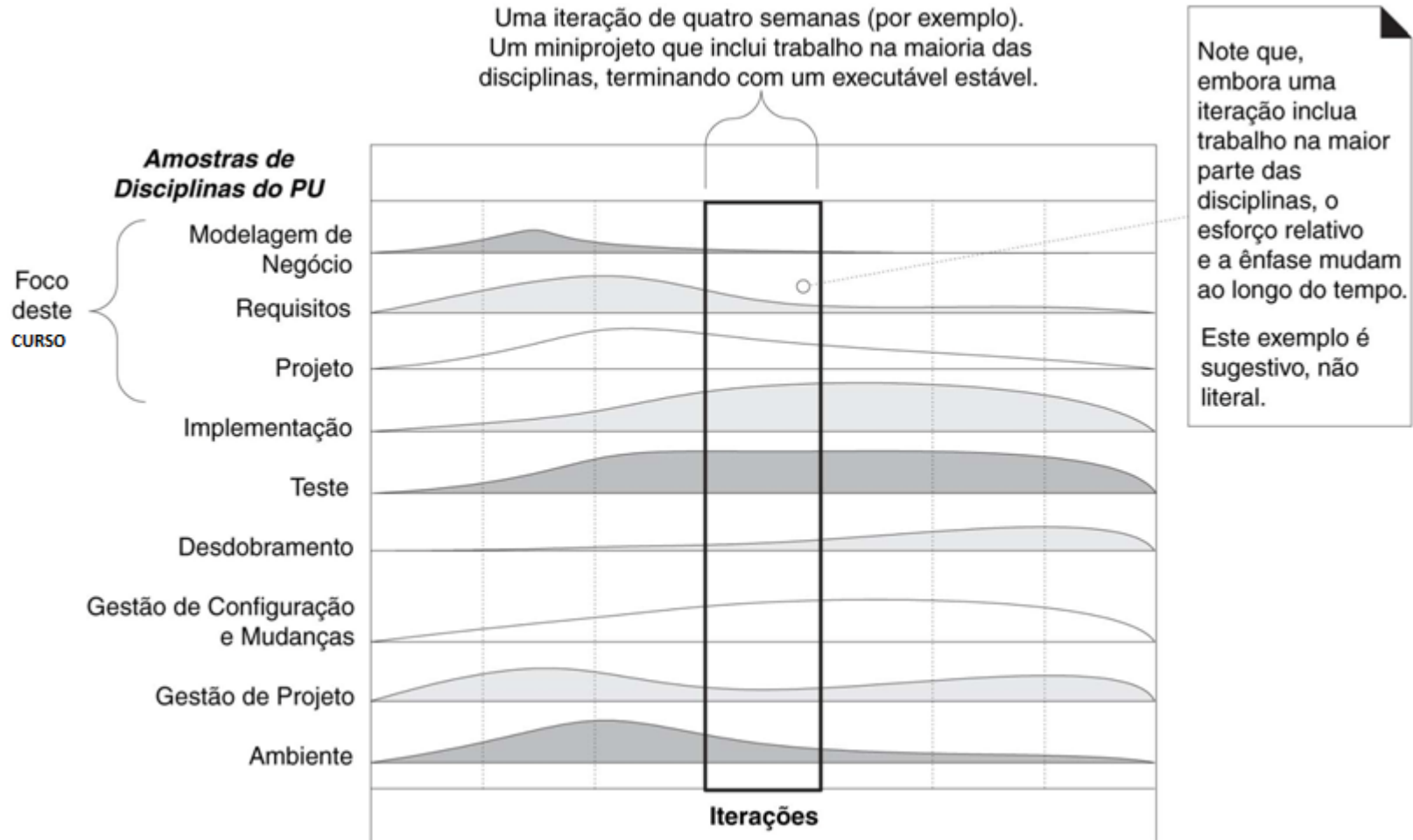
Há outras práticas importantes do Processo Unificado?

- **Melhores práticas:**
 - Atacar o quanto antes elementos de **alto-valor/risco**
 - Envolver os **usuários** continuamente
 - Construir o quanto antes a **arquitetura** principal.
 - Verificar a qualidade com **testes** durante todo o **ciclo de vida**.
 - Aplicar casos de uso quando adequado.
 - Focar nos modelos essenciais **UML**
 - Gerenciar requisitos
 - **Por em prática o gerenciamento de solicitações de mudança**
 - Implementar a **Gestão de Configurações**

Estrutura do Processo Unificado

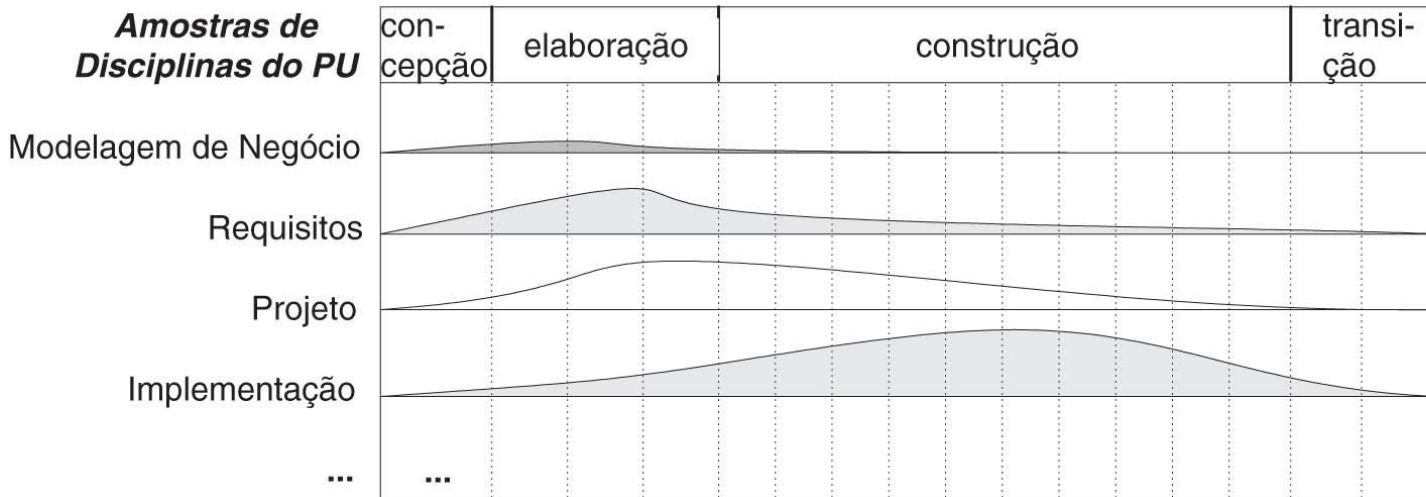


Disciplinas do Processo Unificado



Qual a relação entre disciplina e fases?

Amostras de Disciplinas do PU



O esforço relativo nas disciplinas muda ao longo das fases. Este exemplo é uma sugestão e não deve ser tomado "ao pé da letra".

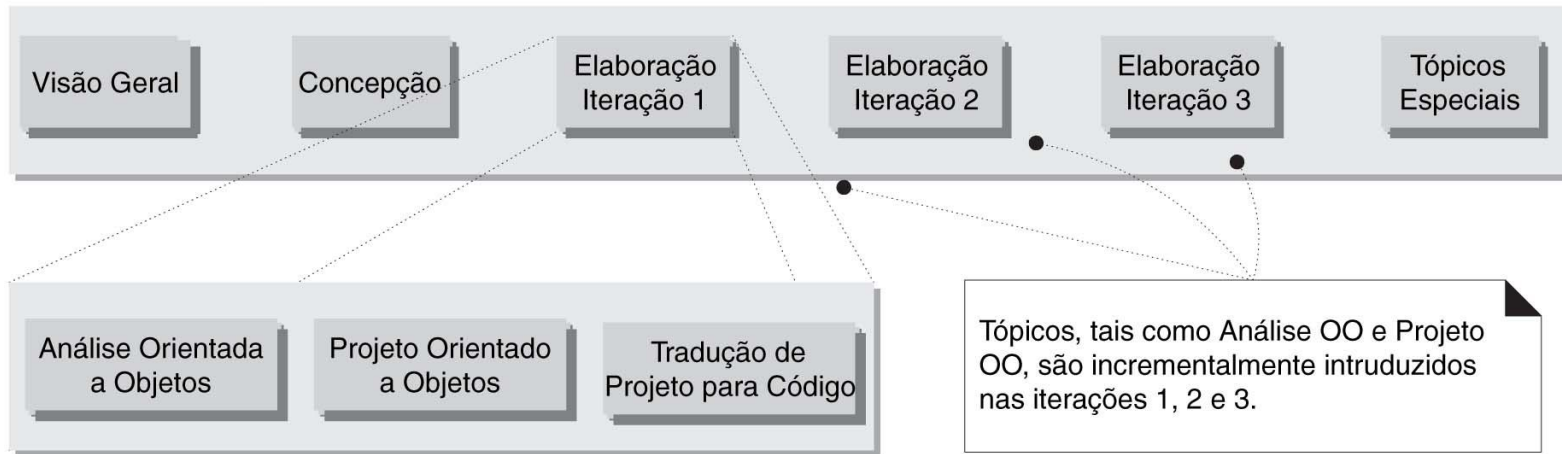
Como a estrutura deste curso é influenciada pelas fases e disciplinas do PU?

- Estudos de caso enfatizam as **fases de concepção e elaboração**
- Artefatos nas **disciplinas**:
 - **Modelagem de negócios**
 - **Requisitos**
 - **Projeto**
- Já que o foco do curso é **A/POO** , **padrões de projeto** e **UML**.

Estrutura do livro e do curso

Nos concentraremos nas **Iterações I, II e III** da fase de elaboração

O Livro



Como personalizar o processo?

- **Quase tudo é opcional no PU**
 - Porém, algumas práticas e princípios são invariáveis
 - Desenvolvimento **iterativo**, orientado ao controle de **riscos** e verificação contínua de **qualidade**
- **Pasta de Desenvolvimento do PU**
 - Artefato curto que documenta as escolhas de **práticas** e **artefatos** usados em um **projeto**.

Pasta de Desenvolvimento do PU

Disciplina	Prática	Artefato	Concepção C1	Elaboração E1..EN	Construção C1..Cn	Transição T1..Tn
Modelagem de Negócios	- Modelagem ágil - Reunião de requisitos	Modelo do Domínio		Iniciar		
	- Reunião de requisitos - Caixa da Visão	Modelo de casos de uso	Iniciar	Refinar		
		Visão	Iniciar	Refinar		
		Especificações Suplementares	Iniciar	Refinar		
		Glossário	Iniciar	Refinar		
Projeto	- Modelagem ágil - Desenvolvimento guiado por testes	Modelo de Projeto		Iniciar	Refinar	
		Documento de arquitetura de software		Iniciar		
		Modelo de dados		Iniciar	Refinar	
Implementação	- Desenvolvimento guiado por testes - Programação em pares - Integração contínua	Modelo de implementação		Iniciar	Refinar	Refinar
Gestão de Projetos	- Scrum	Plano de desenvolvimento de software	Iniciar	Refinar	Refinar	Refinar
Teste	- Testes automatizados	Modelo de Testes		Iniciar	Refinar	
Ambiente		Pasta de Desenvolvimento	Iniciar	Refinar		

Você fica sabendo quando não compreendeu o desenvolvimento iterativo ou PU quando...

- Você tenta de definir a maior parte dos **requisitos** ou **projetos** antes da **implementação**
- Você gasta dias ou semanas em **modelagem UML**
- Você pensa que **Concepção = Requisitos, Elaboração = Projeto, Construção = Implementação**
- Você pensa que usar o **PU** é usar muita documentação

Você fica sabendo quando não compreendeu o desenvolvimento iterativo ou PU quando...

- Você pensa que a **fase de Elaboração** define cuidadosamente os modelos traduzidos em códigos na **Construção**
- Você acredita que a duração adequada de uma **iteração** é de três meses
- Você tenta **planejar** um projeto **em detalhes**

Histórico

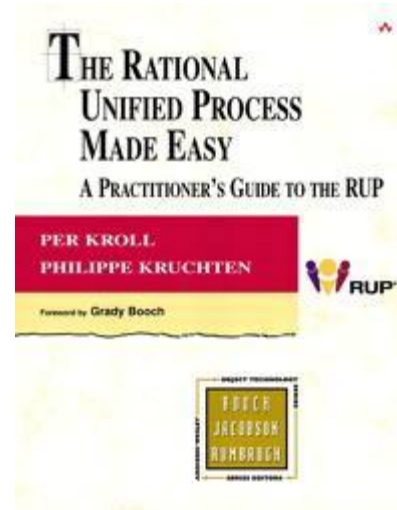
- **Década de 1970**
 - **Tom Gilb** cria e publica o método de desenvolvimento iterativo e incremental chamado **EVO**.
- **Final da década de 1980**
 - O **Departamento de Defesa dos Estados Unidos** substitui os projetos com **ciclo de vida em cascata** para **desenvolvimento iterativo e incremental**
 - **Frederic Brooks** escreve sobre a necessidade de usar métodos iterativos e incrementais em detrimento ao ciclo de vida em cascata
 - **Barry Boehm** um método de desenvolvimento iterativo e incremental com **modelo espiral guiado por riscos**.

Histórico

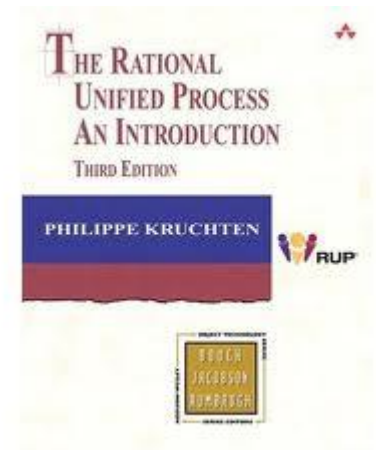
- **Década de 1990**
 - Desenvolvimento iterativo e incremental é reconhecido como sucessor de métodos em cascata
 - Surgimento do **PU, DSDM, Scrum, XP**, etc...

Leituras recomendadas

- Philippe Kruchten e Kroll



- Philippe Kruchten



Leituras recomendadas

- Kent Beck



- Lembre-se que a maior parte das **práticas do XP** é compatível ou idêntica às **práticas do PU**.